

SOMIOD API

Unidade Curricular de Integração de Sistemas – Engenharia Informática

Diogo Miguel Ferreira Gomes
PL1

2022131671
2221439@my.ipleiria.pt

José Pedro Figueiredo Branco
PL4
2022131561
2221424@my.ipleiria.pt

Francisco Xavier Gomes Silva
PL1
2022104332
2191613@my.ipleiria.pt

João Miguel Martins Costa
PL4
2016117476
2160868@my.ipleiria.pt

Abstract— Este documento apresenta o desenvolvimento de um middleware IoT com base no modelo SOMIOD, cujo objetivo principal é poder desempenhar funções de criação, gestão e descoberta de recursos IoT através de uma arquitetura RESTful. O sistema implementado permite a gestão completa de aplicações, contentores, instâncias de conteúdo e subscrições, fornecendo operações CRUD acessíveis via HTTP e MQTT, também podendo ser acedidas através de mecanismos de descoberta baseados em somiod-discovery. A solução desenvolvida integra validação estrutural de dados através de Schemas XML (XSD) e JSON, de modo a garantir a consistência e integridade da informação armazenada, tanto em base de dados relacional como em ficheiros XML. Em adição, foram implementadas aplicações de teste que permitem a interação do utilizador com o sistema de forma intuitiva, tornando possível a criação, atualização e remoção de recursos e a visualização do estado global do sistema. Este projeto demonstra a aplicabilidade de arquiteturas orientadas a serviços no contexto da Internet das Coisas, tornando evidente a importância da interoperabilidade, validação de dados e separação de responsabilidades entre middleware e aplicações cliente. Os resultados obtidos provam que a solução implementada cumpre os requisitos funcionais propostos, constituindo uma base sólida para futuras extensões, principalmente no que toca a notificações assíncronas e integração com dispositivos físicos.

Keywords— *IoT, middleware, RESTful services, SOMIOD, XML, XSD, JSON, somiod-discovery, subscrições, aplicações de teste, instâncias de conteúdo, contentores.*

I. INTRODUÇÃO

A crescente aposta em sistemas de Internet of Things (IoT) tem conduzido no aparecimento de ambientes altamente distribuídos, que são compostos por dispositivos heterogêneos, aplicações e serviços que necessitam de comunicação eficiente, segura e escalável. Neste contexto, os middleware IoT assumem um papel fundamental, acabando por funcionar como uma camada intermédia responsável pela gestão de recursos, normalização da comunicação e

desacoplamento entre quem produz e quem consome os dados.

Um dos principais desafios nesses sistemas consiste na gestão estruturada dos recursos, tais como aplicações, contentores, instâncias de conteúdo e subscrições, bem como na implementação de mecanismos de descoberta, como é o caso do somiod-discovery, e notificação que permitam às aplicações reagir de forma dinâmica a eventos relevantes. De modo a resolver estes desafios, é necessário implementar um middleware que seja capaz de expor uma API bem definida, suportar múltiplos formatos de dados e garantir a validação e consistência da informação trocada.

O trabalho implementado neste projeto tem como foco principal a implementação de um middleware IoT inspirado no modelo SOMIOD, recorrendo a uma arquitetura com base em serviços REST. O sistema desenvolvido permite a criação, leitura, atualização e remoção (métodos CRUD) de aplicações, contentores, instâncias de conteúdo e subscrições, assim como na implementação de um mecanismo de descoberta através de cabeçalhos HTTP específicos. Em adição, o middleware suporta a validação estrutural de dados em JSON e através de Schemas XSD, de modo a garantir a integridade da informação armazenada em formato XML.

Para além do middleware, ainda foi desenvolvida uma aplicação de testes com interface gráfica, que tem como objetivo a interação com a API de forma intuitiva, facilitando a validação funcional do sistema. Esta aplicação possibilita a gestão de vários recursos definidos no middleware e simula cenários típicos de utilização num ambiente IoT, contribuindo deste modo para a verificação prática do funcionamento do sistema.

II. ARQUITETURA DO SISTEMA

O sistema foi validado através de um caso de uso de smarthouse, no qual comandos enviados por uma aplicação

cliente resultam na criação de instâncias de conteúdo no middleware, desencadeando notificações assíncronas que simulam o controle de dispositivos domésticos, como portas inteligentes, iluminação e estores.

A figura 1 representa o esquema da arquitetura do projeto desenvolvido.

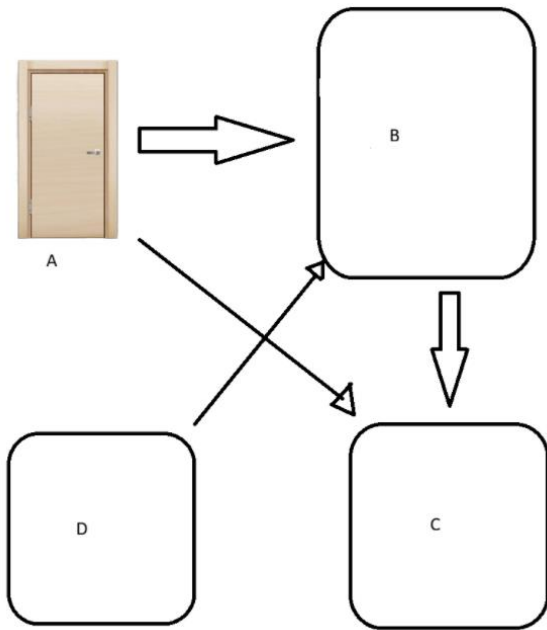


Fig. 1. Esquema da Arquitetura do Projeto

A. Dispositivo IoT – Porta Inteligente (Produto Final)

O Componente A corresponde ao dispositivo IoT final, no esquema está representado como uma porta inteligente, que representa o elemento físico do sistema. Este dispositivo é responsável por executar ações concretas no mundo real, tais como abrir e fechar a porta, alterar estados internos e responder a comandos externos.

A porta inteligente não comunica diretamente com a aplicação do utilizador, sendo totalmente desacoplada da interface gráfica. Em vez disso, interage com o sistema através de mecanismos de comunicação assíncronos, recebendo comandos e enviando notificações sobre o seu estado. Este modelo permite que o dispositivo funcione de forma autónoma e independente da origem dos comandos, facilitando a integração com diferentes aplicações ou serviços no futuro.

B. Middleware SOMIOD

O Componente B corresponde ao middleware SOMIOD, que atua como núcleo central do sistema. Este componente é responsável pela gestão de todos os recursos IoT, incluindo aplicações, contentores, instâncias de conteúdo e subscrições. O middleware expõe uma API Restful que permite o funcionamento dos métodos CRUD destes recursos, garantindo uma estrutura hierárquica e consistente.

Adicionalmente, o SOMIOD implementa um mecanismo de descoberta baseado em cabeçalhos HTTP específicos, sendo este também conhecido como somiod-discovery, permitindo que aplicações externas obtenham de forma dinâmica a lista de recursos disponíveis. As respostas são

fornecidas em formato JSON, facilitando a integração com aplicações cliente.

O middleware suporta o envio de notificações tanto via MQTT como via HTTP. Embora a validação funcional tenha sido demonstrada através de MQTT, o envio por HTTP encontra-se implementado e preparado ao nível do middleware. Desta forma assegura que as alterações relevantes no estado dos recursos possam ser propagadas para os componentes subscritos. Este papel central torna o middleware SOMIOD o principal elemento de coordenação e interoperabilidade do sistema.

C. Sistema de Mensagens MQTT

O Componente C corresponde ao sistema de mensagens MQTT, utilizado como mecanismo de comunicação assíncrona entre o middleware e o dispositivo IoT. O protocolo MQTT encontra-se adequado para ambientes IoT devido à sua leveza, baixo consumo de recursos e modelo baseado em publish/subscribe.

Neste sistema, o MQTT permite a propagação eficiente de eventos e comandos, de forma a garantir que o dispositivo IoT receba apenas as mensagens relevantes para o seu funcionamento. Esta abordagem reduz a ligação entre os componentes e melhora a escalabilidade do sistema, de modo a permitir a adição de novos dispositivos ou serviços sem grandes alterações na arquitetura existente.

D. Aplicação de Testes (Publisher e Subscriber)

As aplicações de teste funcionam como clientes do middleware e têm como principal objetivo validar o funcionamento do sistema de forma prática. A aplicação Publisher permite a criação e gestão de recursos IoT através de pedidos HTTP RESTful, enquanto a aplicação Subscriber recebe e processa notificações assíncronas via MQTT.

Estas aplicações simulam cenários reais de utilização num contexto de smarthouse, permitindo demonstrar o fluxo completo de informação desde a criação de um comando no middleware até à sua execução no dispositivo IoT final. A aplicação Subscriber integra ainda mecanismos de serialização em XML e validação através de XSD, reforçando a robustez e fiabilidade da arquitetura proposta.

III. AVALIAÇÃO

A avaliação do sistema desenvolvido teve como principal objetivo verificar o correto funcionamento do middleware SOMIOD e a sua integração com aplicações cliente e mecanismos de comunicação assíncrona.

A. Test bed

Os testes foram realizados com o Visual Studio 2022 num ambiente de desenvolvimento local, utilizando um computador com sistema operativo Windows, e o versionamento foi feito com GitHub. O middleware foi executado sobre um servidor web local, enquanto o broker MQTT foi igualmente executado localmente. As aplicações Publisher e Subscriber foram desenvolvidas como aplicações desktop com interface gráfica.

B. Validação Funcional

Foram testadas todas as operações **CRUD** disponibilizadas pelo middleware, verificando-se que os recursos são corretamente criados, consultados e removidos. O mecanismo de descoberta somiod-discovery foi igualmente validado.

C. Avaliação das Notificações

O sistema de notificações foi avaliado através da criação de instâncias de conteúdo associadas a subscrições ativas. Em todos os cenários testados, as notificações foram corretamente publicadas via MQTT e recebidas pelas aplicações Subscriber, sendo posteriormente validadas através de XSD.

IV. INTEGRAÇÃO / DESENVOLVIMENTO APLICACIONAL

A. Aplicação Publisher

A aplicação Publisher permite ao utilizador criar e gerir aplicações, contentores, instâncias de conteúdo e subscrições através de uma interface gráfica. A comunicação com o middleware é realizada através de pedidos HTTP RESTful, utilizando os métodos CRUD adequados para cada tipo de recurso.

Entre as funcionalidades disponibilizadas destacam-se:

- criação e remoção de aplicações IoT;
- criação e remoção de containers associados a aplicações;
- criação e remoção de instâncias de conteúdo que representam comandos ou dados enviados para dispositivos de smarthouse;
- criação e remoção de subscrições, definindo os eventos de interesse.

A aplicação Publisher é também responsável por simular o envio de comandos para os dispositivos, como abrir ou fechar uma porta inteligente. Estes comandos são armazenados no middleware sob a forma de instâncias de conteúdo, desencadeando automaticamente notificações para as subscrições registadas.

B. Aplicação Subscriber

A aplicação Subscriber tem como principal função receber notificações emitidas pelo middleware, utilizando o protocolo MQTT como sistema de mensagens. Esta aplicação subscreve os tópicos relevantes no broker MQTT, reagindo sempre que ocorre a criação de novas instâncias de conteúdo associados aos recursos subscritos.

Quando uma notificação é recebida, o payload é inicialmente processado em formato JSON, sendo posteriormente convertido para XML. O documento XML resultante é validado através de um XSD, garantindo que a estrutura da notificação cumpre os requisitos definidos. Apenas notificações estruturalmente válidas são processadas pela aplicação.

Após a validação, a aplicação interpreta o conteúdo da notificação e simula o comportamento do dispositivo IoT correspondente, atualizando o estado visual na interface gráfica. No caso da porta inteligente a aplicação atualiza o estado visual do dispositivo na interface gráfica, refletindo ações como abertura ou fecho da porta.

V. CONCLUSÃO E TRABALHO FUTURO

Neste trabalho foi concebido e implementado um middleware IoT inspirado no modelo SOMIOD, aplicado a um cenário de smarthouse. O sistema desenvolvido suporta a gestão completa de recursos IoT, disponibiliza operações CRUD através de uma API RESTful e integra mecanismos de descoberta e notificação assíncrona.

A utilização de MQTT para envio de notificações, aliada à validação estrutural de dados em JSON e XML, contribui para a robustez e fiabilidade da solução. As aplicações de teste desenvolvidas permitiram validar o funcionamento do sistema e demonstrar um fluxo completo de utilização.

Como trabalho futuro, poderá ser considerada a implementação de um consumidor dedicado de notificações via HTTP, a integração com dispositivos IoT físicos reais, melhorias ao nível da segurança e a realização de testes de desempenho em cenários de maior escala.

VI. REFERENCES

- [1] Miguel Mira da Silva, "Integração de Sistemas de Informação", FCA, 2003.
- [2] Michael Rosen, Boris Lublinsky, Kevin T. Smith, Marc J. Balcer, "Applied SOA: Service-Oriented Architecture and Design Strategies", Wiley Publishing Inc., 2008, ISBN 978-0-470-22365-9.
- [3] Julia Lerman, "Programming Entity Framework", O'Reilly Media, Inc., 2009, ISBN-13: 978-0-596-52028-1.
- [4] Chris Bowen, Richard Crane, Steve Resnick, "Essential Windows Communication Foundation (WCF): For .NET Framework 3.5", Addison-Wesley Professional, 2008, ISBN-13: 978-0321440068.
- [5] Mark Masse, "REST API Design Rulebook, Designing Consistent RESTful Web Service Interfaces", O'Reilly Media, 2011, ISBN10: 1-4493-1050-8.
- [6] Jon Flanders, RESTful .NET, Build and Consume RESTful Web Services with .NET 3.5, O'Reilly Media, 2008, ISBN 978-0-596-51920-9.
- [7] Apontamentos teóricos e fichas práticas de Integração de Sistemas.

APPENDIX

Appendix A

Aplicações

GET ALL: curl -location

```
'http://localhost:58066/api/somiod/' \
--header 'somiod-discovery: application'
```

GET By Name:

```
curl -location 'http://localhost:58066/api/somiod/ola'
```

POST: curl -location 'http://localhost:58066/api/somiod/' \

```
--header 'Accept: application/json' \
```

```
--header 'Content-Type: application/json' \
```

```
--data '{
```

```
  "resource-name": "app_teste"
```

```
}',
```

```
PUT:      curl      --location      --request      PUT
'http://localhost:58066/api/somiod/app_teste' \
--header 'Accept: application/json' \
--header 'Content-Type: application/json' \
--data '{
  "resource-name": "app_teste"
}'
```

```
DELETE:   curl      --location      --request      DELETE
'http://localhost:58066/api/somiod/app_teste' \
--header 'Accept: application/json' \
--header 'Content-Type: application/json'
```

Containers

```
GET      ALL:      curl      --location
'http://localhost:58066/api/somiod/myapp' \
--header 'Accept: application/json' \
--header 'Content-Type: application/json' \
--header 'somiod-discovery: container'
```

```
GET      By      Name:      curl      --location
'http://localhost:58066/api/somiod/myapp/c1'
```

POST:

```
curl --location 'http://localhost:58066/api/somiod/myapp' \
--header 'Accept: application/json' \
--header 'Content-Type: application/json' \
--data '{
  "resource-name": "new_container"
}'
```

```
PUT:      curl      --location      --request      PUT
'http://localhost:58066/api/somiod/myapp/new_containe
r' \
--header 'Accept: application/json' \
--header 'Content-Type: application/json' \
--data '{
  "application-resource-name": "ola"
}'
```

```
DELETE:   curl      --location      --request      DELETE
'http://localhost:58066/api/somiod/ola/new_cont'
```

Content-Instances

```
GET      ALL:      curl      --location
'http://localhost:58066/api/somiod/' \
--header 'Accept: application/json' \
--header 'Content-Type: application/json' \
--header 'somiod-discovery: content-instance'
```

```
GET      ALL      By      App:      curl      --location
'http://localhost:58066/api/somiod/smarthouse/' \
--header 'Content-Type: application/json' \
--header 'somiod-discovery: content-instance' \
--data "
```

```
GET      ALL      By      Container:  curl      --location
'http://localhost:58066/api/somiod/smarthouse/door' \
--header 'Content-Type: application/json' \
--header 'somiod-discovery: content-instance' \
--data "
```

```
GET      By      Name:      curl      --location
'http://localhost:58066/api/somiod/myapp/c1/ci\_teste'
```

```
POST:      curl      --location
'http://localhost:58066/api/somiod/myapp/c1' \
--header 'Content-Type: application/json' \
--header 'Accept: application/json' \
--data '{
  "resource-name": "ci_teste",
  "content-type": "text/plain",
  "res-type": "content-instance",
  "content": "Teste"
}'
```

```
DELETE:   curl      --location      --request      DELETE
'http://localhost:58066/api/somiod/myapp/c1/ci_teste'
```

Subscriptions

```
GET      ALL      Geral:      curl      --location
'http://localhost:58066/api/somiod/' \
--header 'Content-Type: application/json' \
--header 'Accept: application/json' \
--header 'somiod-discovery: subscription' \
--data "
```

```
GET ALL Por Aplicação: curl --location
'http://localhost:58066/api/somiod/smarthouse/' \
--header 'Content-Type: application/json' \
--header 'somiod-discovery: subscription' \
--data "
```

```
"resource-name": "sub-teste",
"evt": 1,
"res-type": "subscription",
"endpoint": "http://localhost:6000/notify"
}'
```

```
GET ALL Por Container: curl --location
'http://localhost:58066/api/somiod/smarthouse/door' \
--header 'Content-Type: application/json' \
--header 'somiod-discovery: subscription' \
--data "
```

```
DELETE: curl --location --request DELETE
'http://localhost:58066/api/somiod/myapp/c1/subs/sub-
teste'
```

```
GET By Name: curl --location
'http://localhost:58066/api/somiod/smarthouse/door/subs/doorEvents'
```

Appendix B

CRUDs de Application, Container e Content Instance –
Diogo Gomes e Francisco Silva.

```
POST: curl --location
'http://localhost:58066/api/somiod/myapp/c1' \
--header 'Content-Type: application/json' \
--header 'Accept: application/json' \
--data '{
```

CRUD Subscription, notificações, XSD, aplicações de teste
Publisher e Subscriber - João Costa e José Branco

Usamos os seguintes “Nuget’s packages” para correr as
soluções Somiod (SomiodSubscriber/SomiodPublisher) -
RestSharp;

Newtonsoft.Json; M2Mqtt para todas as soluções.